Project - High Level Design

On

Marketing Content Generator

Course Name: **Generative AI**

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	YASH YADAV	EN22CS3011113
2	AYUSH MISHRA	EN22EL301020
3	VISHAL	EN22CS3011091
4	ASHUTOSH KUMAR	EN22EL301019
5	HARSH MANDLOI	EN22ME304035
6	AYUSH MODI	EN21CS301196

Group Name: 13D10

Project Number: GAI-26

Industry Mentor Name: Mr. Yash

University Mentor Name: Prop Divya Kumawat

Academic Year: 2025-2026

Table of Contents

1. Introduction

This High Level Design (HLD) document provides a detailed internal view of the Marketing Content Generator, a GenAI-powered application. It describes the design of each module, class, method, and data structure to ensure developers can understand, maintain, or extend the system. The system focuses on modularity, statelessness per request, and extensibility.

1.1 Scope of the Document

The scope includes the detailed design of:

1. LLM Engine: The core interface for Groq API interactions.
2. Content Vector Store: Management of the ChromaDB vector database.
3. Prompt Templates: Structured engineering for various marketing content types.
4. Streamlit UI: The user interface and session management.
5. File Exporter: Utility functions for document serialization.

1.2 Intended Audience

This document is intended for developers, system architects, and maintainers who need to understand the internal logic, database schema, API contracts, and inter-module communication of the project.

1.3 System Overview

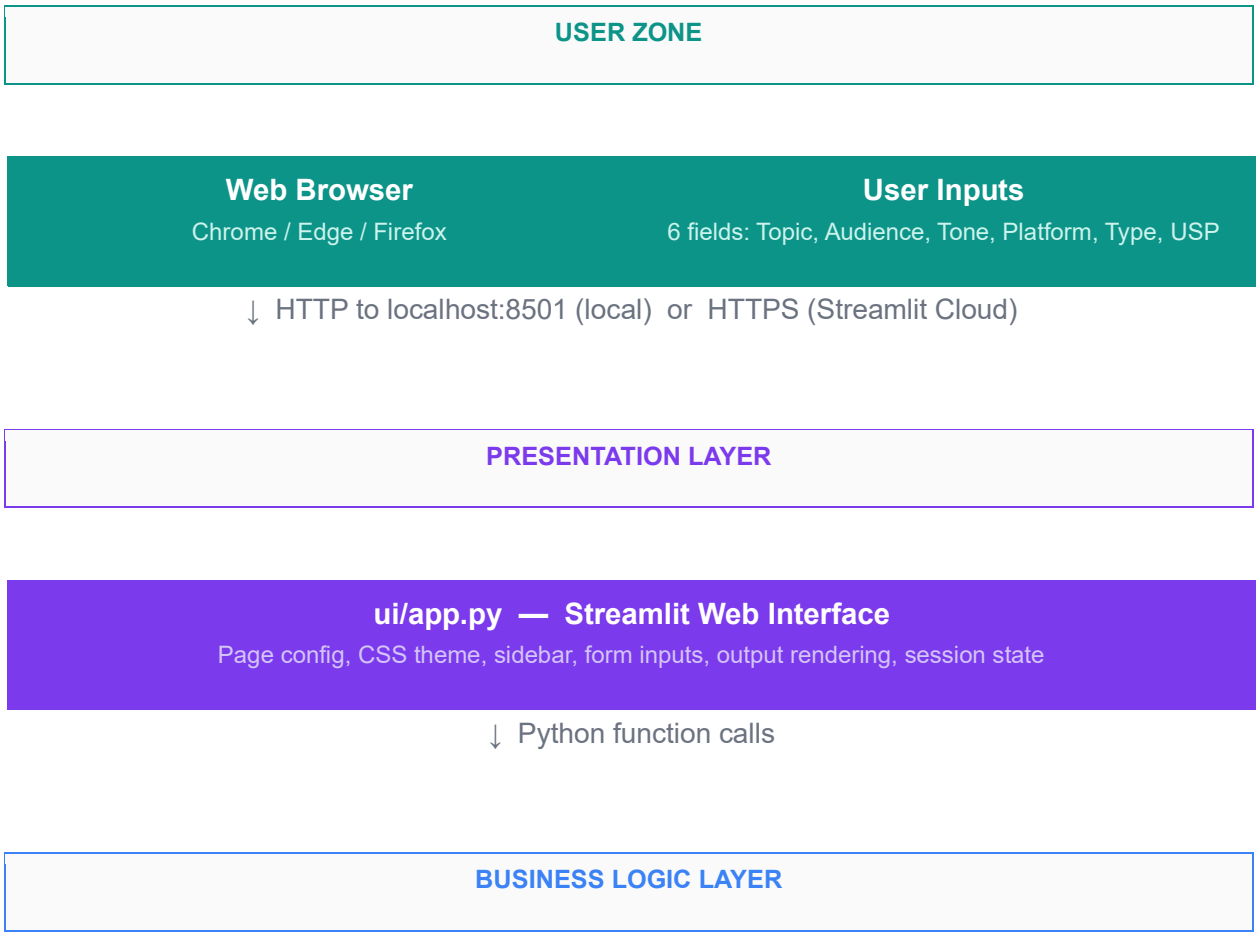
The Marketing Content Generator is a Python-based application using **Groq LLM** for text generation and **ChromaDB** for semantic memory. It utilizes a 5-layer unidirectional pipeline where data flows from user input through the prompt engine, LLM, and vector store, and finally back to the UI.

2. System Design

2.1 Application Design

The application is structured into five distinct layers:

1. **L1 (UI):** ui/app.py – Collects inputs and renders outputs.
2. **L2 (Prompts**): prompts/templates.py – Builds structured LLM prompts.
3. **L3 (Engine):** core/llm_engine.py – Handles Groq API communication.
4. **L4 (Vector DB):** vectordb/store.py – Manages embeddings and persistence.
5. **L5 (Utils):** utils/exporter.py – Formats content for .txt and .docx.



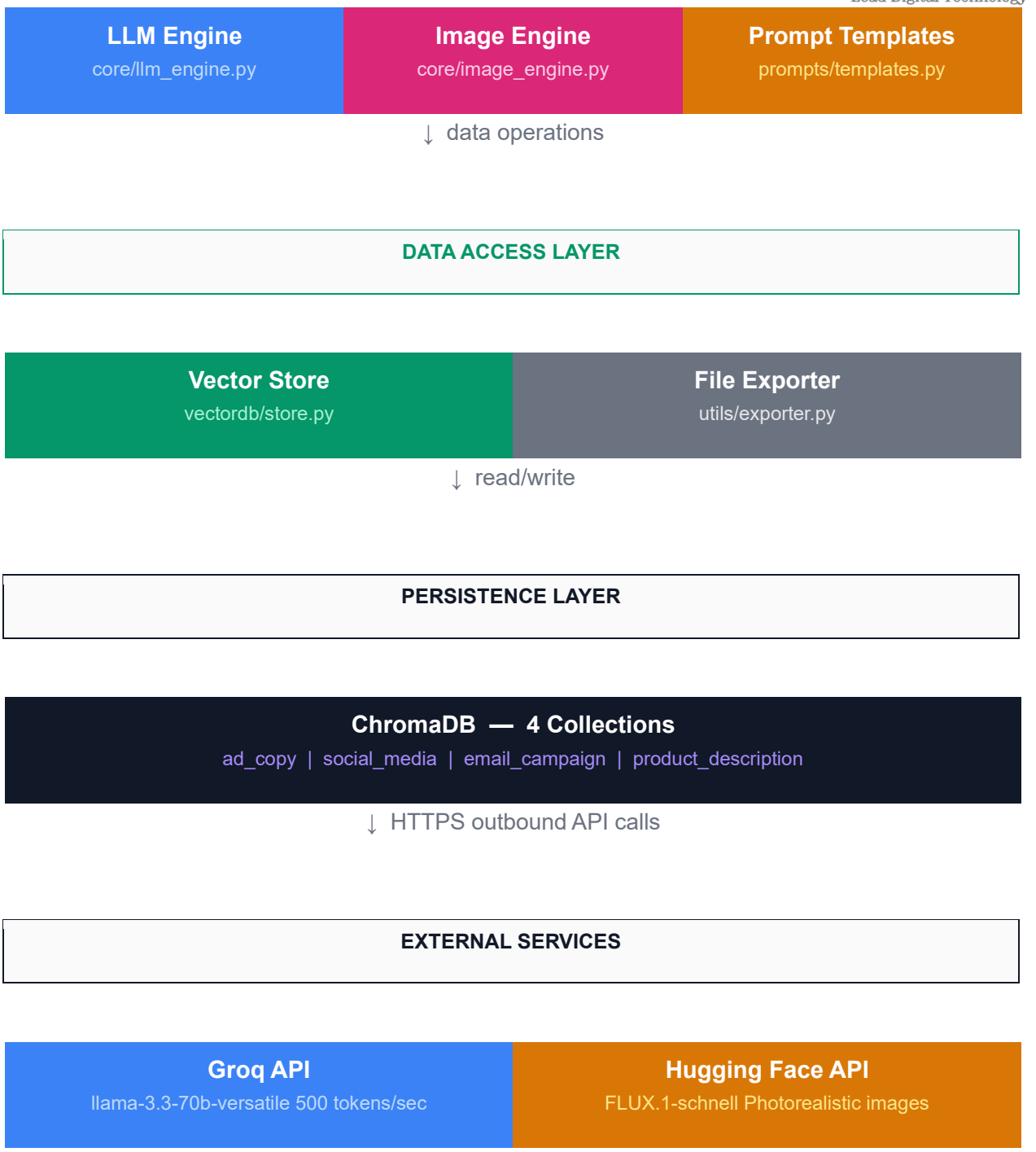
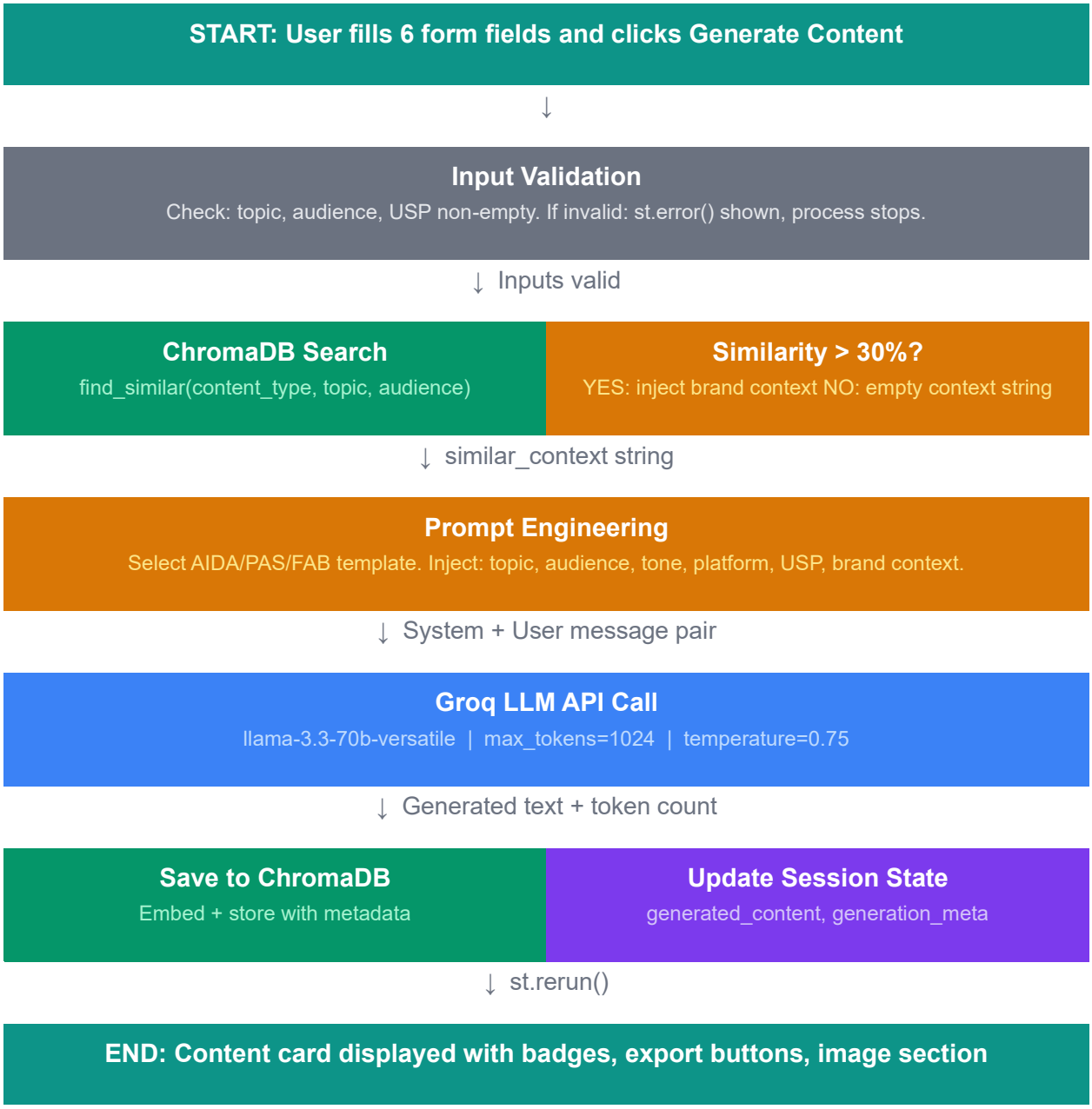


Fig.1: **System Architecture Diagram — 5 Layer Model**

2.2 Process Flow

The system supports two primary process flows. Both are triggered by user interaction in the Streamlit interface.

Process Flow 1 — Text Content Generation



Process Flow 2 — AI Image Generation

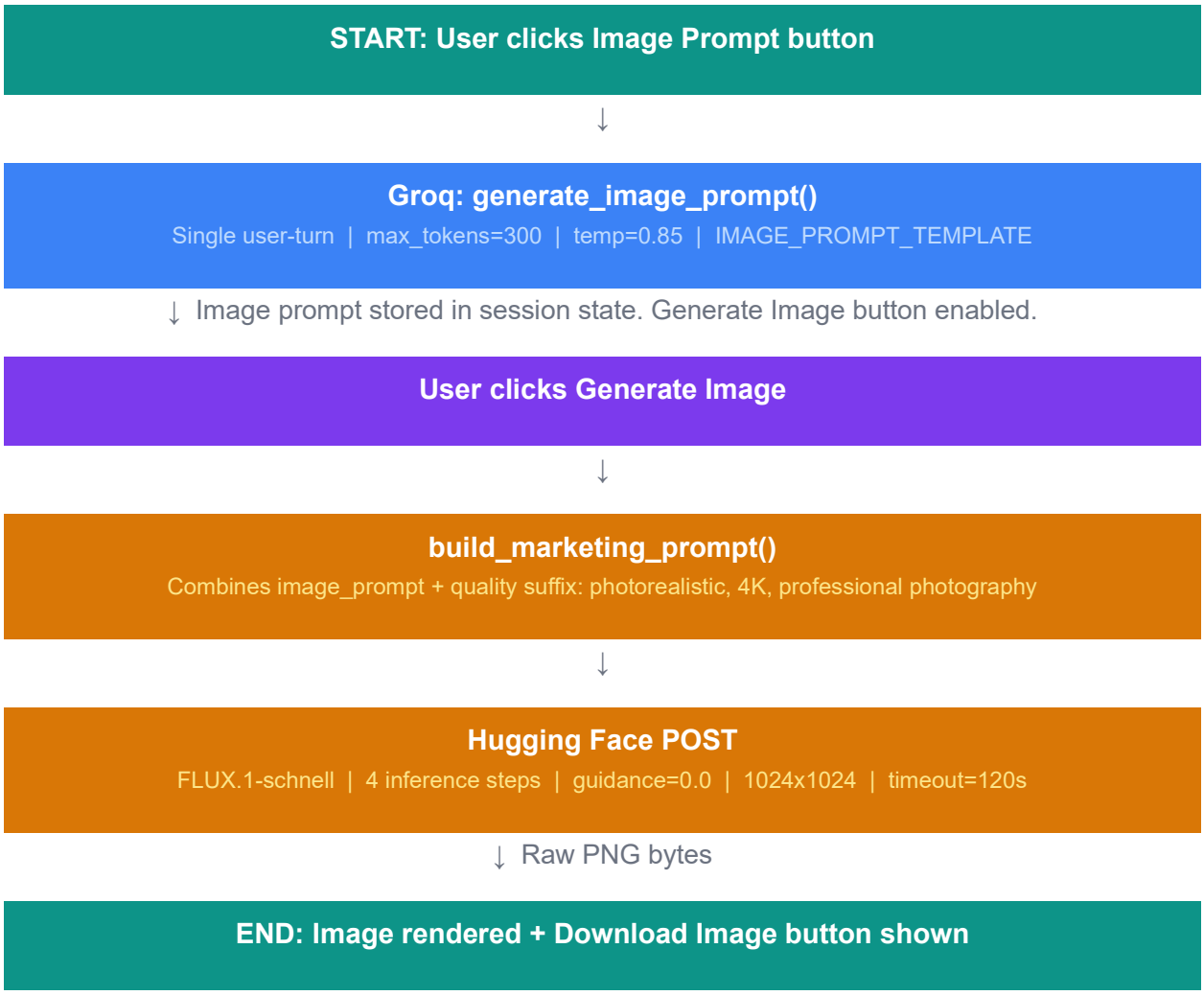


Fig.2: Process Flow

2.3 Information Flow

This table shows how data transforms as it moves through the system from raw user inputs to final outputs.

Stage	Input	Output
User Input	Raw form fields (strings)	Validated Python variables: 6 typed strings
Vector Search	topic + audience strings	similar_context: formatted past brand content or empty string
Prompt Building	6 inputs + similar_context	Structured prompt: system + user messages (500-800 tokens total)
LLM Generation	Structured prompt	Generated content string (300-800 tokens) + integer token count
Vector Storage	Content + metadata	ChromaDB doc: UUID id + vector[384] + metadata dict
Image Prompt Gen	content_type, topic, tone, platform	Detailed image prompt string (150-300 words)
Image Generation	Image prompt + quality suffix	Raw PNG bytes (1024x1024, approx 2-4MB)
Export	Generated content + metadata	Plain text string (.txt) or binary bytes (.docx)

2.4 Components Design

Each component has a single, well-defined responsibility. Components communicate only through explicit function calls with no shared global state between modules.

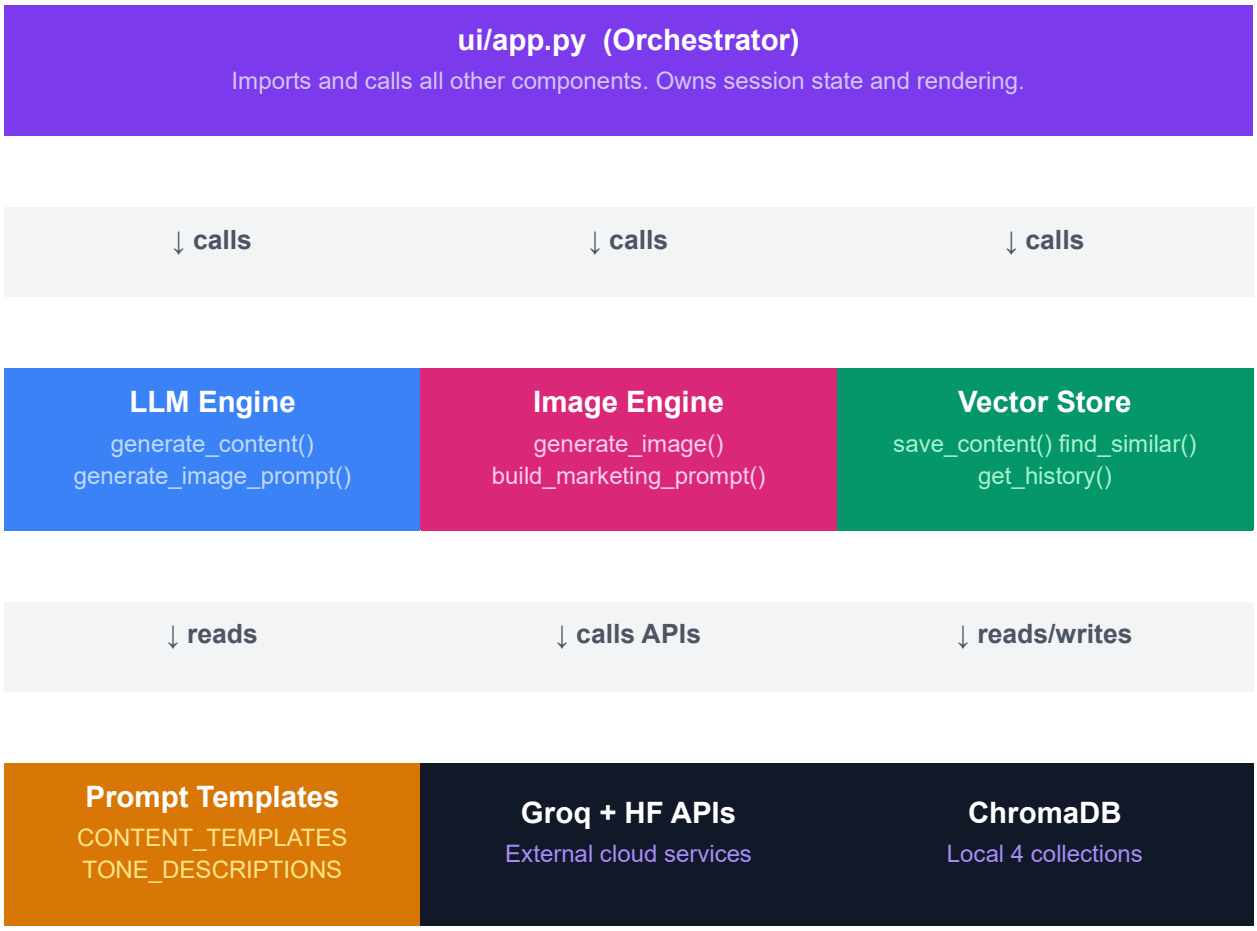


Fig.3: **Component Interaction Diagram**

Component	Pattern	Key Methods	Caching
LLMEngine	Singleton (cached)	generate_content(), generate_image_prompt()	@st.cache_resource
ImageEngine	Singleton (cached)	generate_image(), build_marketing_prompt()	@st.cache_resource
ContentVectorStore	Singleton (cached)	save_content(), find_similar(), get_history(), delete_all()	@st.cache_resource
Prompt Templates	Pure data module	No methods — constants only	N/A (static)
File Exporter	Stateless utility	export_to_txt(),	N/A (pure functions)

Component	Pattern	Key Methods	Caching
		export_to_docx()	

2.5 Key Design Considerations

- **Caching**: Uses @st.cache_resource to instantiate services once, avoiding redundant API client or DB initializations.
- **Error Handling**: All API calls are wrapped in try/except blocks to isolate faults.
- **Similarity Threshold**: A 30% similarity filter is applied to prevent irrelevant context injection.
- **Statelessness**: Logic modules remain stateless; data persists only in the database or the UI session.

2.6 . API Catalogue

API / SDK	Endpoint / Method	Purpose
Groq SDK	chat.completions.create	Text generation and image prompting
ChromaDB	collection.query	Semantic search for similar past content
ChromaDB	collection.add	Storing documents with automated embeddings
Python-Docx	doc.save(buf)	In-memory generation of Word documents

3 Data Design

3.1 Data Model

The system uses ChromaDB as its primary data store with no relational database. ChromaDB stores documents as vector embeddings alongside metadata. Four

independent collections are used to prevent cross-type contamination in semantic search.

ChromaDB Collection Structure

ChromaDB PersistentClient
./vectordb/chroma_store/ (local) | /tmp/chroma_store (Streamlit Cloud)

ad_copy	social_media	email_campaign	product_description

Document Schema

Field	Type	Source	Description
id	string (UUID4)	uuid.uuid4()	Primary key — unique document identifier
document	string	Constructed	Topic + Audience + generated content — embedding source
embedding	float[384]	ChromaDB auto	all-MiniLM-L6-v2 semantic vector
metadata.topic	string	User input	Product or campaign topic
metadata.audience	string	User input	Target audience description
metadata.tone	string	User input	Selected brand tone name
metadata.platform	string	User input	Selected platform or channel
metadata.timestamp	string	datetime.now()	Generation time: YYYY-MM-DD HH:MM:SS
metadata.content_type	string	Derived	One of 4 content type strings

3.2 Data Access Mechanism

All data access goes through the Content Vector Store class. No other component accesses ChromaDB directly. This ensures the access layer can be replaced without touching business logic.

Operation	Method	Description
Create	save_content()	Embeds document and stores with metadata. Returns UUID.
Read (semantic)	find_similar()	Cosine similarity query — returns context string for prompt injection.
Read (all)	get_history()	Retrieves all documents sorted by timestamp descending.
Delete	delete_all()	Removes all documents from one or all collections.
Count	collection.count()	Guard: checks collection has docs before querying.

Similarity Threshold

Only results with cosine similarity > 30% (1 - cosine_distance > 0.30) are injected into the prompt. This threshold was chosen empirically to prevent irrelevant past content from corrupting new generations.

3.3 Data Retention Policies

- **Storage**: Every generation is automatically persisted to the local vector store upon successful creation.
- **Manual Deletion**: Users have full control over their data and can delete all documents from specific collections or the entire database via the "Manage Data" expander in the UI.

- **Session-Based**: While the database is persistent, the active display is managed via Streamlit session state, which resets when the browser session ends.

3.4 Data Migration

- **Initialization**: On startup, the system calls _get_or_create() for all collections, which is idempotent—it either opens existing data or creates new structures if none are found.

- **Portability**: Since the database is a local folder, it can be excluded from version control via .gitignore to keep user-specific history private and prevent repository bloating.

4. Interfaces

4.1 User Interface (Streamlit)

- **Form-Based Input**: A dedicated sidebar and main panel collect inputs like Content Type, Topic, Audience, Tone, Platform, and USP.
- **Dynamic Output Panel**: Renders the generated marketing copy alongside metadata badges (model name, tokens used) and action buttons for image prompts or file downloads.
- **History Sidebar**: Provides a scrollable list of past generations with filtering capabilities by content type.

4.2 External & Internal APIs

- **Groq Cloud API**: The primary interface for text generation using the llama-3.3-70b-versatile model via the Groq Python SDK.
- **ChromaDB Internal API**: Manages the local vector store through a Persistent Client for semantic search and content persistence.
- **File Export Interface**: Uses python-docx and standard string formatting to provide downloadable .docx and .txt files directly in the browser.

5.State and Session Management

- **Session Persistence**: Utilizes st.session_state to store generated content, generation meta, and image prompt across user interactions without losing data on page reruns.
- **Service Caching**: Employs @st.cache_resource to instantiate the LLM Engine and Content Vector Store only once per server process, optimizing performance and resource usage.
- **Reset Logic**: New generation triggers automatically clear previous image prompts to ensure context remains relevant to the current output.

6.Caching

The application implements a high-efficiency caching strategy to optimize performance and reduce redundant API or database initializations.

- **Service Singleton Pattern**: The LLM Engine and Content Vector Store are instantiated exactly once per Streamlit server process using the @st.cache_resource decorator.
- **Resource Optimization**: This strategy avoids re-reading the .env configuration file and re-initializing the Groq client on every user interaction.
- **Database Efficiency**: It prevents the system from repeatedly re-opening the ChromaDB Persistent Client connection, which would otherwise slow down content generation and retrieval.
- **Manual Cache Invalidation**: The cache is programmatically cleared via st.cache_resource.clear() when a user selects "Clear All History".
- **Fresh Initialization**: Clearing the cache forces a fresh initialization of the Content Vector Store, ensuring the application accurately reflects the now-empty state of the ChromaDB collections.

7. Non-Functional Requirements

7.1 Security Aspects

- **Secret Protection**: Sensitive credentials like the GROQ_API_KEY are stored in a .env file that is strictly excluded from version control via .gitignore.
- **Data Locality**: The ChromaDB vector store runs entirely on the user's local machine, ensuring brand data and history are not stored on external servers.
- **Input Validation**: The system validates all user inputs (topic, audience, USP) before processing to prevent empty or malformed API requests.

7.2 Performance Aspects

- **High-Speed Inference**: By utilizing the Groq API, the application achieves inference speeds of up to 500 tokens per second.
- **Memory Efficiency**: Export functions for .txt and .docx files operate using in-memory buffers (io.BytesIO), avoiding unnecessary local disk I/O operations.
- **Modular Isolation**: The unidirectional pipeline ensures that errors in one module do not cascade to others, maintaining system stability during high usage.

8. References

Reference	Details
Groq API Documentation	https://console.groq.com/docs
Groq Deprecations	https://console.groq.com/docs/deprecations
Hugging Face Inference API	https://huggingface.co/docs/api-inference
FLUX.1-schnell Model Card	https://huggingface.co/black-forest-labs/FLUX.1-schnell
ChromaDB Documentation	https://docs.trychroma.com
Streamlit Documentation	https://docs.streamlit.io
Streamlit Session State	https://docs.streamlit.io/develop/api-reference/caching-and-state
sentence-transformers	https://www.sbert.net
AIDA Framework	Elmo Lewis, 1898 — Attention, Interest, Desire, Action

Reference	Details
PAS Framework	Dan Kennedy — Problem, Agitate, Solution
FAB Framework	Features, Advantages, Benefits — product description standard
GitHub Repository	https://github.com/Ayush-js/marketing_gen
Live Application	https://marketinggen-hdxstvx33cpb868akgjryi.streamlit.app